

Technical Report: Fine-Tuning a Route Replanning Model

AdaptiveRoute POC

August 25, 2026

Abstract

This report documents the design, training, evaluation, and final selection of the specialized route replanning model used in the AdaptiveRoute proof of concept. The approach combines a prescriptive optimization layer, implemented with Pyomo and HiGHS, with supervised fine-tuning of a neural route-candidate policy. The fine-tuned model is not treated as a mathematical optimizer and does not replace the solver. It is used as a fast structured candidate generator, protected by deterministic validation, repair logic, and a Pyomo + HiGHS fallback. The selected final version is the LoRA adapter at `outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5`, based on `Qwen/Qwen2.5-7B-Instruct`.

1 Executive Summary

The fine-tuning track produced a specialized model for generating JSON route replanning candidates under operational disruptions. The final selected model, referred to as v5, reached a 94.4% feasible-candidate rate on a fixed 1,000-example benchmark sampled from `data/training_curriculum_100k/sft_test.jsonl`. It improved over the earlier v3 and v4 checkpoints and outperformed the later v6 continuation, which slightly regressed.

The key architectural decision is to keep optimization and learning separated:

- Pyomo + HiGHS is the prescriptive source of truth.
- The fine-tuned LLM is a route-candidate policy.
- Deterministic validation decides whether a candidate can be accepted.
- Repair and solver fallback remain mandatory.

This makes the system suitable for an agentic AI architecture where a reasoning-capable orchestrator can delegate route candidate generation to the fine-tuned specialist model.

2 Project Objective

The goal of the fine-tuning module was to train a specialist model that receives:

- a structured routing scenario;
- an existing route plan;
- an operational event, such as a blocked arc or unavailable customer;
- vehicle capacities, customer demands, and route constraints;

and returns a replanned route candidate in a strict JSON format.

The proof of concept focuses on synthetic CVRP instances, primarily with 8 customers and 2 vehicles. The target behavior is operational feasibility, not proof of optimality. A feasible generated plan is useful only if it passes deterministic validation.

3 Theoretical Foundation

3.1 Vehicle Routing and CVRP

The Vehicle Routing Problem was introduced by Dantzig and Ramser in the classical truck dispatching formulation [1]. In the Capacitated Vehicle Routing Problem, a fleet of vehicles with limited capacity must serve a set of customers from a depot while minimizing total travel cost or distance.

CVRP is a combinatorial optimization problem. For realistic instances, exact optimization can become expensive, and practical systems often combine mathematical programming, heuristics, and learned policies. In this project, the learned model is not positioned as an exact solver. It is trained to imitate high-quality prescriptive decisions generated by a mathematical optimization engine.

3.2 Prescriptive Optimization

The prescriptive layer answers the question: given a scenario and a disruption, what route plan should be executed? Pyomo was selected because it allows explicit algebraic modeling in Python and keeps the optimization model inspectable, testable, and version-controlled [6]. HiGHS was selected as the open-source solver because it provides high-performance optimization routines for LP, MIP, and QP problems and has a Python interface suitable for local experimentation [7].

This combination supports the intended technical message: the route optimization model is explicit and auditable, rather than hidden inside a black-box model.

3.3 Transformer-Based Language Models

The base model is a causal Transformer model [2]. The attention mechanism is useful for this task because the model must condition route decisions on relationships among customers, vehicles, capacity constraints, blocked arcs, and the previous plan.

An instruction-tuned model was preferred because the task is contract-driven: the model must follow a structured prompt and produce valid JSON.

3.4 LoRA and QLoRA

Full fine-tuning of a 7B-parameter model was not practical for the local hardware target. Instead, the project used parameter-efficient fine-tuning with LoRA. LoRA freezes the original pretrained weights and injects trainable low-rank matrices into selected layers, drastically reducing the number of trainable parameters [3].

The training setup also used QLoRA-style 4-bit loading. QLoRA enables fine-tuning large language models by keeping the base model quantized while training LoRA adapters, using techniques such as NF4 quantization and double quantization [4].

Therefore, the full base model was not trained. The base `Qwen/Qwen2.5-7B-Instruct` model remained frozen; only LoRA adapter weights were updated.

4 Selected Base Model

The selected base model was `Qwen/Qwen2.5-7B-Instruct`. The selection criteria were practical:

- open-weight model suitable for local experimentation;
- approximately 7.61B parameters;
- 28 Transformer layers;

- modern architecture features, including RoPE, SwiGLU, RMSNorm, and Grouped Query Attention;
- strong instruction-following behavior for structured output;
- compatibility with Hugging Face Transformers, TRL, PEFT, and bitsandbytes.

The model is intended to serve as a specialized routing policy. It should not be the same model responsible for user-facing reasoning. In the planned agentic AI architecture, a stronger reasoning model can handle conversation, orchestration, and explanation, while the fine-tuned model serves route-specialist agents or tools.

5 Prescriptive Optimization Component

The prescriptive engine was implemented with Pyomo + HiGHS. It has three roles:

1. solve synthetic CVRP instances to generate supervised labels;
2. validate whether LLM-generated candidates satisfy deterministic constraints;
3. provide fallback plans when generated candidates are invalid or missing.

A simplified CVRP objective is:

$$\min \sum_{k \in K} \sum_{i \in N} \sum_{j \in N, j \neq i} d_{ij} x_{ijk}$$

subject to customer coverage:

$$\sum_{k \in K} \sum_{i \in N, i \neq j} x_{ijk} = 1 \quad \forall j \in C$$

capacity:

$$\sum_{j \in C} q_j y_{jk} \leq Q_k \quad \forall k \in K$$

flow conservation:

$$\sum_{j \in N, j \neq i} x_{ijk} = \sum_{j \in N, j \neq i} x_{jik} \quad \forall i \in N, \forall k \in K$$

and blocked-arc exclusion:

$$x_{ijk} = 0 \quad \forall (i, j) \in B, \forall k \in K$$

where N is the node set, C is the customer set, K is the vehicle set, d_{ij} is travel distance, q_j is customer demand, Q_k is vehicle capacity, B is the blocked-arc set, x_{ijk} indicates whether vehicle k traverses arc (i, j) , and y_{jk} indicates whether vehicle k serves customer j .

The implementation also includes depot start/end rules, route continuity, subtour elimination, and event-specific constraints.

6 Dataset Generation

6.1 Generation Methodology

The datasets were generated synthetically to provide scale, repeatability, and reliable labels. The generation pipeline was:

1. generate a CVRP instance with depot, customers, demands, vehicles, capacities, and distance matrix;
2. solve the base route plan with Pyomo + HiGHS;
3. apply an operational event, such as a blocked arc or unavailable customer;
4. re-solve the disrupted scenario with the solver;
5. validate the replanned route;
6. serialize the scenario, event, and target output into compact SFT JSONL;
7. convert compact SFT rows into chat-message JSONL for instruction fine-tuning.

This approach makes the LLM learn from prescriptive optimization outputs while preserving a deterministic oracle for auditing and runtime validation.

6.2 Generated Datasets

Dataset	Total	Train	Validation	Test	Purpose
training_compact_30k	30000	24000	3000	3000	Initial compact curriculum.
training_capacity_tight_50k_parallel	50000	40000	5000	5000	Emphasis on capacity pressure.
training_blocked_arc_20k_parallel	20000	16000	2000	2000	Emphasis on blocked arcs.
training_curriculum_100k	100000	80000	10000	10000	Consolidated curriculum dataset.
training_error_driven_20k	20000	16000	2000	2000	Continuation data focused on observed errors.
training_error_driven_10k_v6	10000	8000	1000	1000	Additional continuation after v5.

The corresponding chat-format datasets were stored under `data/chat_training_*`. The main split strategy was 80% training, 10% validation, and 10% test. Local audits found zero invalid rows in the principal datasets used for training.

7 Fine-Tuning Strategy

7.1 Training Algorithm

The selected training algorithm was supervised fine-tuning with LoRA/QLoRA. Training used:

- Hugging Face Transformers for model loading;

- TRL **SFTTrainer** for supervised fine-tuning;
- PEFT for LoRA adapter management;
- bitsandbytes for 4-bit model loading.

The optimization target is causal language-model loss over the assistant response tokens. In practice, the model learns to map a structured route replanning prompt to the expected JSON route candidate.

7.2 Final LoRA Configuration

Parameter	Value
Base model	Qwen/Qwen2.5-7B-Instruct
Method	LoRA / QLoRA-style loading
PEFT type	LORA
Task type	CAUSAL_LM
Rank r	16
<code>lora_alpha</code>	32
<code>lora_dropout</code>	0.05
Bias	none
Target modules	<code>q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj</code>
Quantization	4-bit NF4 with double quantization
Compute dtype	bfloat16

This configuration trains adapter matrices in attention and MLP projection modules. The original dense weights of the base model remain frozen.

8 Training Methodology and Parameters

Training was conducted in stages. Each continuation was evaluated against a fixed benchmark before deciding whether to keep training or freeze the current model.

Version	Train rows	Eval rows	Steps	Batch	LR	Time	Notes
v3	24000	500	3000	1×8 accum.	$5 \cdot 10^{-5}$	5h53m	First stable full training on the 30K compact dataset.
v4	56000	1000	2300	8	$1.7 \cdot 10^{-5}$	3h44m	Hard-data continuation using SDPA and checkpoint resume.
v5	16000	1000	1500	8	$1 \cdot 10^{-5}$	2h22m	Error-driven continuation. Selected final version.
v6	8000	500	800	8	$7 \cdot 10^{-6}$	1h18m	Additional continuation; slightly regressed.

The stabilized training setup used:

- `--max-seq-length`2048 for v4/v5/v6 continuations;
- `--bf16`;
- `--bnb-compute-dtype`bf16;
- `--optim`adamw_torch;
- `--lr-scheduler-type`cosine;
- `--max-grad-norm`0.3;
- `--no-eval-during-train`;
- `dataset_num_proc`=1;
- `dataloader_num_workers`=0.

The representative v5 training command was:

```
uv run python scripts/train_lora.py \
  --model-id Qwen/Qwen2.5-7B-Instruct \
  --adapter-path outputs/models/adaptiveroute-qwen2_5-7b-lora-hard70k-v4-b8-sdpa-resume700 \
  --dataset-dir data/chat_training_error_driven_20k \
  --output-dir outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5 \
  --max-seq-length 2048 \
  --epochs 1 \
  --max-steps 1500 \
  --learning-rate 1e-5 \
  --warmup-steps 50 \
  --max-grad-norm 0.3 \
  --lr-scheduler-type cosine \
  --per-device-batch-size 8 \
  --gradient-accumulation-steps 1 \
  --train-limit 16000 \
  --eval-limit 1000 \
  --bf16 \
  --bnb-compute-dtype bfloat16 \
  --optim adamw_torch \
  --save-steps 100 \
  --logging-steps 10 \
  --no-eval-during-train
```

9 Training Feedback

9.1 Training Metrics

The logs show that loss quickly converged to the 0.26–0.28 range after the initial learning stage. Mean token accuracy stabilized around 0.89. These metrics indicate that the model learned the response format and common routing patterns, but they are not sufficient to prove operational correctness. The primary evaluation criterion remained route feasibility.

Version	Train loss	Final token accuracy	Tokens processed	Runtime
v3	0.2796	0.8922	$1.614 \cdot 10^7$	21190s
v4	0.2659	0.8915	$1.241 \cdot 10^7$	13470s
v5	0.2654	0.8911	$8.081 \cdot 10^6$	8510s
v6	0.2653	0.8917	$4.307 \cdot 10^6$	4700s

9.2 Operational Evaluation

All candidate models were evaluated on the same 1,000-example sample from `data/training_curriculum_100k/sft_test.jsonl`. The main metric was feasible-candidate rate. Exact match was tracked but treated as secondary because CVRP can have multiple valid route plans for the same scenario.

Model	Valid JSON	Feasible	Exact match	Capacity violations	Blocked-arc violations
v3	100.0%	91.7%	4.6%	72	11
v4	100.0%	92.9%	5.8%	61	10
v5	100.0%	94.4%	5.6%	46	9
v6	100.0%	94.0%	5.4%	50	10

The main result is that v5 improved feasibility and reduced capacity violations relative to v4. The later v6 continuation did not improve the fixed benchmark and was not selected.

10 Limitations Found

10.1 Model Limitations

- The model does not guarantee mathematical feasibility or optimality.
- Residual violations remain. On the 1,000-example benchmark, v5 still produced 46 capacity violations and 9 blocked-arc violations.
- Exact match is low, although this is partly expected for CVRP because multiple feasible solutions can exist.
- Training data is synthetic; no real traffic, fleet telemetry, driver constraints, service times, or geospatial road-network data were used in this stage.
- The primary training scope was small CVRP instances, approximately 8 customers.

10.2 Infrastructure Limitations

- Long-running generation and training jobs exposed native failures, including `exitstatus139`, double-free errors, and a `torch.nn.Module.train` runtime issue.
- FlashAttention was not successfully installed because the local environment used Python 3.12, Torch with CUDA 13.0, and an `nvcc/CUDA` stack without a compatible prebuilt wheel.
- The stable workaround was to use SDPA, reduce preprocessing parallelism, disable evaluation during training, and use `adamw_torch`.

11 Refinement Strategy

The next refinements should target operational feasibility, not marginal loss reduction. The recommended strategy is:

1. integrate v5 into the agentic routing module behind a deterministic validator;
2. implement repair for simple capacity, missing-customer, and blocked-arc violations;
3. use Pyomo + HiGHS fallback when validation or repair fails;
4. generate new error-driven datasets from actual validator failures;
5. train new continuations only if they beat v5 on the fixed benchmark;
6. test best-of-N route candidates with deterministic selection;
7. expand synthetic data to more customers, more vehicles, time windows, multiple depots, asymmetric costs, and richer disruption types;
8. include public VRP/CVRP benchmarks when moving beyond the current POC scope;
9. revisit FlashAttention or Unsloth only after stabilizing a compatible CUDA stack.

12 Final Version and Runtime Contract

The selected model is:

`outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5`

The stable operational alias is:

`outputs/models/adaptiveroute-routing-policy-current`

The recommended runtime contract is:

```
scenario + base_plan + event
-> LoRA v5 candidate generation
-> JSON parsing
-> deterministic route validation
-> repair if needed
-> Pyomo + HiGHS fallback if still invalid
-> final response
```

The final accepted response should include candidate source, validation result, rejected violations if any, route distance, vehicle loads, and final route JSON.

13 Training and Test Environment

14 Time Spent

Item	Observed value
Operating system	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-138-generic
CPU	Intel Core i9-14900K, 32 logical CPUs
GPU	NVIDIA GeForce RTX 3090
VRAM	24576 MiB
NVIDIA driver	580.173.02
Compute capability	8.6
Python	3.12.3 via uv
Torch	2.13.0, CUDA 13.0
Transformers	5.15.1
TRL	1.10.0
PEFT	0.20.0
bitsandbytes	0.50.1
datasets	5.0.1
Pyomo	6.10.1
HiGHS/highspy	1.15.1

Activity	Observed time	Notes
Initial smoke training	3m24s	20 steps; validated the basic pipeline.
v3 30K training	5h53m	First stable full training run.
v4 hard 70K continuation	3h44m	Continued from checkpoint with SDPA.
v5 error-driven 20K continuation	2h22m	Selected final model.
v6 error-driven 10K continuation	1h18m	Did not outperform v5.
1,000-example prediction generation	20–25min per model	Observed during v5/v6 evaluation runs.
Synthetic dataset generation	Long chunked/parallel jobs	Monitored with screen , systemd-inhibit , and tee logs.

15 Conclusion

The selected approach is technically defensible for a five-day proof of concept. The prescriptive Pyomo + HiGHS layer provides deterministic labels, validation, and fallback. The LoRA/QLoRA fine-tuning process specializes an instruction-following 7B model into a route-candidate generator. The final model decision was based on operational feasibility, not only training loss.

The v5 adapter is the current best stopping point. It achieved the highest feasible-candidate rate among evaluated models and reduced the most important violation categories. The next engineering step should be integration into the LangGraph-based agentic AI module, with a validator, repair layer, and solver fallback. Further training should be driven by runtime validation failures rather than by generating more synthetic data without a measured error target.

References

- [1] G. B. Dantzig and J. H. Ramser. The Truck Dispatching Problem. *Management Science*, 6(1):80–91, 1959. <https://doi.org/10.1287/mnsc.6.1.80>

- [2] A. Vaswani et al. Attention Is All You Need. arXiv:1706.03762, 2017. <https://arxiv.org/abs/1706.03762>
- [3] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [4] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023. <https://arxiv.org/abs/2305.14314>
- [5] Qwen Team. Qwen2.5 Technical Report and Qwen2.5-7B-Instruct model card. <https://arxiv.org/abs/2412.15115> and <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>
- [6] M. L. Bynum et al. Pyomo – Optimization Modeling in Python. Third Edition, Springer, 2021. <https://www.pyomo.org/citing-pyomo>
- [7] HiGHS. High-performance open-source software for linear optimization. <https://highs.dev/>